

Лабораторная работа: Поиск элемента

1.0

Generated by Doxygen 1.17.0

1	Результаты экспериментов	1
1.1	Сравнение всех методов поиска	1
1.2	Зависимость времени поиска от числа уникальных ключей	3
1.3	Влияние размера хэш-таблицы на коллизии и время поиска	4
1.4	Сравнение хэш-функций	5
1.5	Тепловая карта загрузки хэш-таблицы	6
1.6	Общие выводы по лабораторной работе	8
2	Class Index	9
2.1	Class List	9
3	File Index	11
3.1	File List	11
4	Class Documentation	13
4.1	BSTNode Struct Reference	13
4.1.1	Detailed Description	13
4.1.2	Constructor & Destructor Documentation	14
4.1.2.1	BSTNode()	14
4.1.3	Member Data Documentation	14
4.1.3.1	key	14
4.1.3.2	left	14
4.1.3.3	right	14
4.1.3.4	values	14
4.2	BSTree Class Reference	15
4.2.1	Detailed Description	15
4.2.2	Constructor & Destructor Documentation	16
4.2.2.1	BSTree()	16
4.2.2.2	~BSTree()	16
4.2.3	Member Function Documentation	16
4.2.3.1	clear() [1/2]	16
4.2.3.2	clear() [2/2]	16
4.2.3.3	find()	16
4.2.3.4	findNode()	17
4.2.3.5	insert() [1/2]	17
4.2.3.6	insert() [2/2]	17
4.2.4	Member Data Documentation	17
4.2.4.1	root	17
4.3	DistributionStats Struct Reference	18
4.3.1	Member Data Documentation	18
4.3.1.1	avgLoad	18
4.3.1.2	collisions	18
4.3.1.3	loadDistribution	18
4.3.1.4	loadFactor	18

4.3.1.5	maxLoad	18
4.3.1.6	nonEmptyBuckets	18
4.3.1.7	totalBuckets	18
4.4	Flight Class Reference	19
4.4.1	Detailed Description	19
4.4.2	Constructor & Destructor Documentation	19
4.4.2.1	Flight() [1/2]	19
4.4.2.2	Flight() [2/2]	20
4.4.3	Member Function Documentation	20
4.4.3.1	getKey()	20
4.4.4	Friends And Related Symbol Documentation	20
4.4.4.1	operator<<	20
4.4.4.2	operator>>	20
4.4.5	Member Data Documentation	20
4.4.5.1	airline	20
4.4.5.2	arrivalDate	21
4.4.5.3	arrivalTime	21
4.4.5.4	flightNumber	21
4.4.5.5	passengers	21
4.5	FlightRecord Struct Reference	21
4.5.1	Detailed Description	21
4.5.2	Member Data Documentation	22
4.5.2.1	airline	22
4.5.2.2	arrivalDate	22
4.5.2.3	arrivalTime	22
4.5.2.4	flightNumber	22
4.5.2.5	passengers	22
4.6	HashFunctionWrapper Struct Reference	22
4.6.1	Member Data Documentation	23
4.6.1.1	description	23
4.6.1.2	func	23
4.6.1.3	name	23
4.7	HashTable Class Reference	23
4.7.1	Detailed Description	24
4.7.2	Constructor & Destructor Documentation	24
4.7.2.1	HashTable()	24
4.7.2.2	~HashTable()	24
4.7.3	Member Function Documentation	24
4.7.3.1	clear()	24
4.7.3.2	find()	25
4.7.3.3	getBucketSizes()	25
4.7.3.4	getCollisions()	25
4.7.3.5	hash()	25

4.7.3.6 insert()	26
4.7.4 Member Data Documentation	26
4.7.4.1 buckets	26
4.7.4.2 collisions	26
4.7.4.3 tableSize	26
4.8 HashTableWithHash< HashFunc > Class Template Reference	26
4.8.1 Constructor & Destructor Documentation	27
4.8.1.1 HashTableWithHash()	27
4.8.2 Member Function Documentation	27
4.8.2.1 find()	27
4.8.2.2 getCollisions()	27
4.8.2.3 insert()	27
4.8.3 Member Data Documentation	27
4.8.3.1 buckets	27
4.8.3.2 collisions	27
4.8.3.3 hashFunc	28
4.8.3.4 tableSize	28
4.9 RBNode Struct Reference	28
4.9.1 Detailed Description	28
4.9.2 Constructor & Destructor Documentation	29
4.9.2.1 RBNode()	29
4.9.3 Member Data Documentation	29
4.9.3.1 color	29
4.9.3.2 key	29
4.9.3.3 left	29
4.9.3.4 parent	29
4.9.3.5 right	29
4.9.3.6 values	30
4.10 RBTree Class Reference	30
4.10.1 Detailed Description	31
4.10.2 Constructor & Destructor Documentation	31
4.10.2.1 RBTree()	31
4.10.2.2 ~RBTree()	31
4.10.3 Member Function Documentation	31
4.10.3.1 clear() [1/2]	31
4.10.3.2 clear() [2/2]	31
4.10.3.3 find()	31
4.10.3.4 findNode()	32
4.10.3.5 fixInsert()	32
4.10.3.6 insert()	32
4.10.3.7 rotateLeft()	33
4.10.3.8 rotateRight()	33
4.10.4 Member Data Documentation	33

4.10.4.1 root	33
5 File Documentation	35
5.1 benchmark_hash_functions.cpp File Reference	35
5.1.1 Detailed Description	36
5.1.2 Function Documentation	36
5.1.2.1 djbHash()	36
5.1.2.2 generateFlights()	37
5.1.2.3 getHashFunctions()	37
5.1.2.4 getTestKeys()	37
5.1.2.5 jsHash()	37
5.1.2.6 main()	37
5.1.2.7 polyHash()	38
5.1.2.8 rsHash()	38
5.1.3 Variable Documentation	38
5.1.3.1 HASH_TABLE_SIZE	38
5.1.3.2 NUM_SEARCHES	38
5.1.3.3 RANDOM_SEED	39
5.1.3.4 TOTAL_SIZE	39
5.1.3.5 UNIQUE_KEYS_COUNT	39
5.2 benchmark_hash_size.cpp File Reference	39
5.2.1 Detailed Description	40
5.2.2 Запуск	40
5.2.3 Выходные данные	40
5.2.4 Function Documentation	41
5.2.4.1 generateFlights()	41
5.2.4.2 generateTableSizes()	41
5.2.4.3 getTestKeys()	41
5.2.4.4 isPrime()	42
5.2.4.5 main()	42
5.2.4.6 measureSearchTime()	43
5.2.5 Variable Documentation	43
5.2.5.1 NUM_SEARCHES	43
5.2.5.2 RANDOM_SEED	43
5.2.5.3 TOTAL_SIZE	43
5.2.5.4 UNIQUE_KEYS_COUNT	43
5.3 benchmark_unique_keys.cpp File Reference	44
5.3.1 Detailed Description	44
5.3.2 Запуск	45
5.3.3 Выходные данные	45
5.3.4 Function Documentation	46
5.3.4.1 generateFlights()	46
5.3.4.2 main()	46

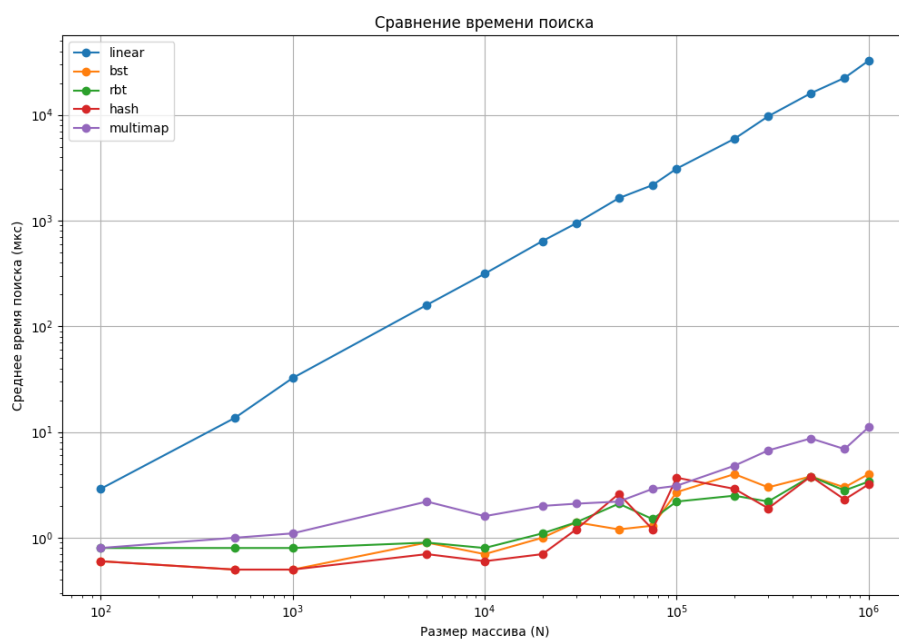
5.3.4.3 measureSearchTime()	46
5.3.4.4 selectTestKeys()	47
5.3.5 Variable Documentation	47
5.3.5.1 HASH_TABLE_SIZE	47
5.3.5.2 NUM_SEARCHES	47
5.3.5.3 RANDOM_SEED	47
5.3.5.4 TOTAL_SIZE	48
5.4 BSTree.cpp File Reference	48
5.4.1 Detailed Description	48
5.5 BSTree.hpp File Reference	48
5.5.1 Detailed Description	48
5.6 BSTree.hpp	49
5.7 Flight.cpp File Reference	49
5.7.1 Detailed Description	49
5.7.2 Function Documentation	49
5.7.2.1 operator<<()	49
5.7.2.2 operator>>()	50
5.8 Flight.hpp File Reference	50
5.8.1 Detailed Description	50
5.9 Flight.hpp	50
5.10 generate_data.cpp File Reference	51
5.10.1 Detailed Description	52
5.10.2 Запуск	52
5.10.3 Выходные данные	52
5.10.4 Function Documentation	52
5.10.4.1 gen()	52
5.10.4.2 main()	52
5.10.4.3 randomAirline()	53
5.10.4.4 randomDate()	53
5.10.4.5 randomFlightNumber()	53
5.10.4.6 randomPassengers()	53
5.10.4.7 randomTime()	54
5.10.4.8 saveToCSV()	54
5.10.5 Variable Documentation	54
5.10.5.1 AIRLINES	54
5.10.5.2 PREFIXES	54
5.10.5.3 RANDOM_SEED	54
5.10.5.4 UNIQUE_FLIGHT_NUMBERS	55
5.11 hash_heatmap.cpp File Reference	55
5.11.1 Detailed Description	55
5.11.2 Выходные данные	56
5.11.3 Для построения тепловой карты:	56
5.11.4 Function Documentation	56

5.11.4.1 analyzeDistribution()	56
5.11.4.2 generateFlights()	56
5.11.4.3 main()	56
5.11.4.4 saveBucketLoadCSV()	56
5.11.4.5 saveTextReport()	56
5.11.5 Variable Documentation	56
5.11.5.1 HASH_TABLE_SIZE	56
5.11.5.2 RANDOM_SEED	57
5.11.5.3 TOTAL_SIZE	57
5.11.5.4 UNIQUE_KEYS_COUNT	57
5.12 HashTable.cpp File Reference	57
5.12.1 Detailed Description	57
5.13 HashTable.hpp File Reference	57
5.13.1 Detailed Description	57
5.14 HashTable.hpp	58
5.15 main.cpp File Reference	58
5.15.1 Detailed Description	59
5.15.2 Function Documentation	59
5.15.2.1 getUniqueKeys()	59
5.15.2.2 linearSearch()	59
5.15.2.3 loadFromCSV()	60
5.15.2.4 main()	60
5.16 RBTree.cpp File Reference	60
5.16.1 Detailed Description	60
5.17 RBTree.hpp File Reference	60
5.17.1 Detailed Description	61
5.17.2 Enumeration Type Documentation	61
5.17.2.1 Color	61
5.18 RBTree.hpp	61
Предметный указатель	63

Глава 1

Результаты экспериментов

1.1 Сравнение всех методов поиска



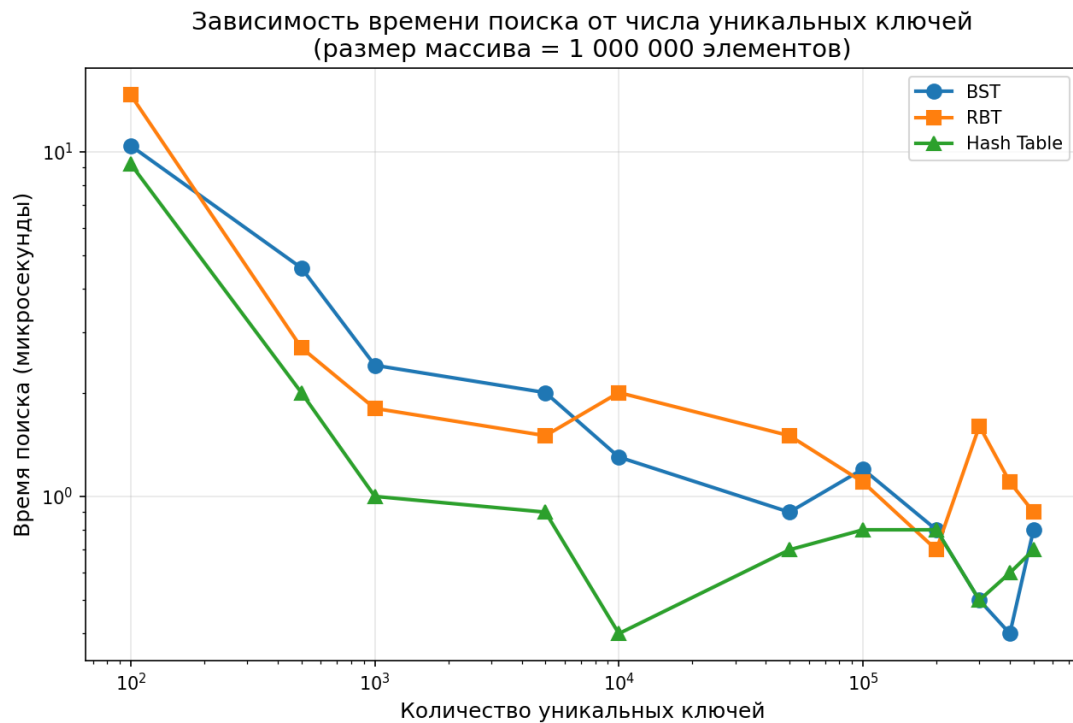
Анализ результатов:

- Линейный поиск (красный): Время растёт линейно $O(n)$. При $N = 1\,000\,000$ достигает 32513 мкс (32.5 мс) на один поиск. непригоден для больших массивов.
- BST (синий): Время почти не зависит от N (4 мкс при $N=1$ млн), так как дерево содержит только 2000 уникальных ключей. Дубликаты хранятся в векторах, не создавая новые узлы.
- RBTree (зелёный): Результаты аналогичны BST (3.4 мкс), но немного медленнее из-за хранения цвета и родителя. Преимущество RBT проявится на данных с большим числом уникальных ключей.

- HashTable (фиолетовый): Постоянное время $O(1)$ (3.2 мкс), коллизий мало (475 при $N=1$ млн). Лучший выбор для поиска по точному совпадению.
- `std::multimap` (оранжевый): Хранит каждый дубликат отдельно, поэтому при $N=1$ млн содержит 1 млн узлов. Время поиска 11.1 мкс (высота дерева около 20 сравнений).

Вывод: Для данных с повторяющимися ключами группировка дубликатов в узле (BST, RBT, HashTable) даёт значительный выигрыш по сравнению со стандартным `multimap`. Линейный поиск следует использовать только для очень малых массивов (менее 1000 элементов).

1.2 Зависимость времени поиска от числа уникальных ключей

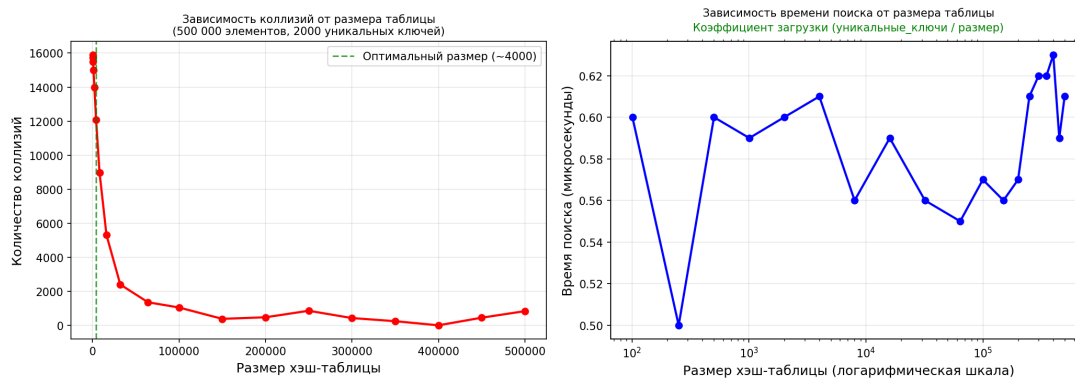


Анализ:

- При фиксированном размере массива (1 млн) менялось количество уникальных ключей от 100 до 500 000.
- BST и RBT: время растёт логарифмически $O(\log U)$.
- HashTable: время остаётся константным (размер таблицы 200003).
- При $U = 500\,000$ (50 процентов уникальных ключей) BST требует около 19 сравнений, HashTable 2-3.

Вывод: Чем больше уникальных ключей, тем больше преимущество хэш-таблицы. При малом числе уникальных ключей (U менее 10 000) разница между структурами незначительна.

1.3 Влияние размера хэш-таблицы на коллизии и время поиска

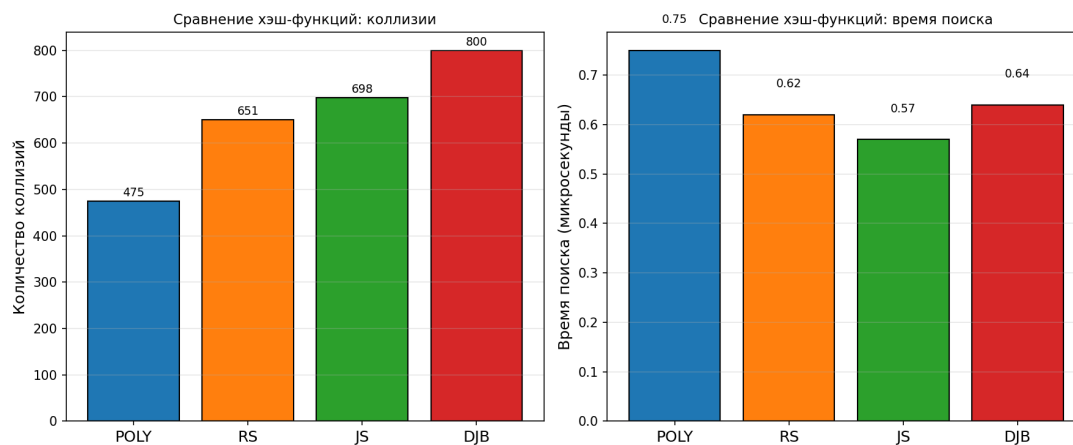


Анализ:

- Фиксированный массив: 500 000 элементов, 2000 уникальных ключей.
- При малом размере таблицы (101) коэффициент загрузки около 20, много коллизий, время поиска высокое.
- Оптимальный размер: примерно 4000 (2 умножить на число уникальных ключей), коллизии минимальны, время стабильно.
- При дальнейшем увеличении размера (200 000 и более) коллизии стремятся к 475, время не улучшается, но память тратится впустую.

Вывод: Размер хэш-таблицы должен быть в 2-4 раза больше числа уникальных ключей. Слишком маленькая таблица ведёт к многим коллизиям, слишком большая к избыточному расходу памяти.

1.4 Сравнение хэш-функций

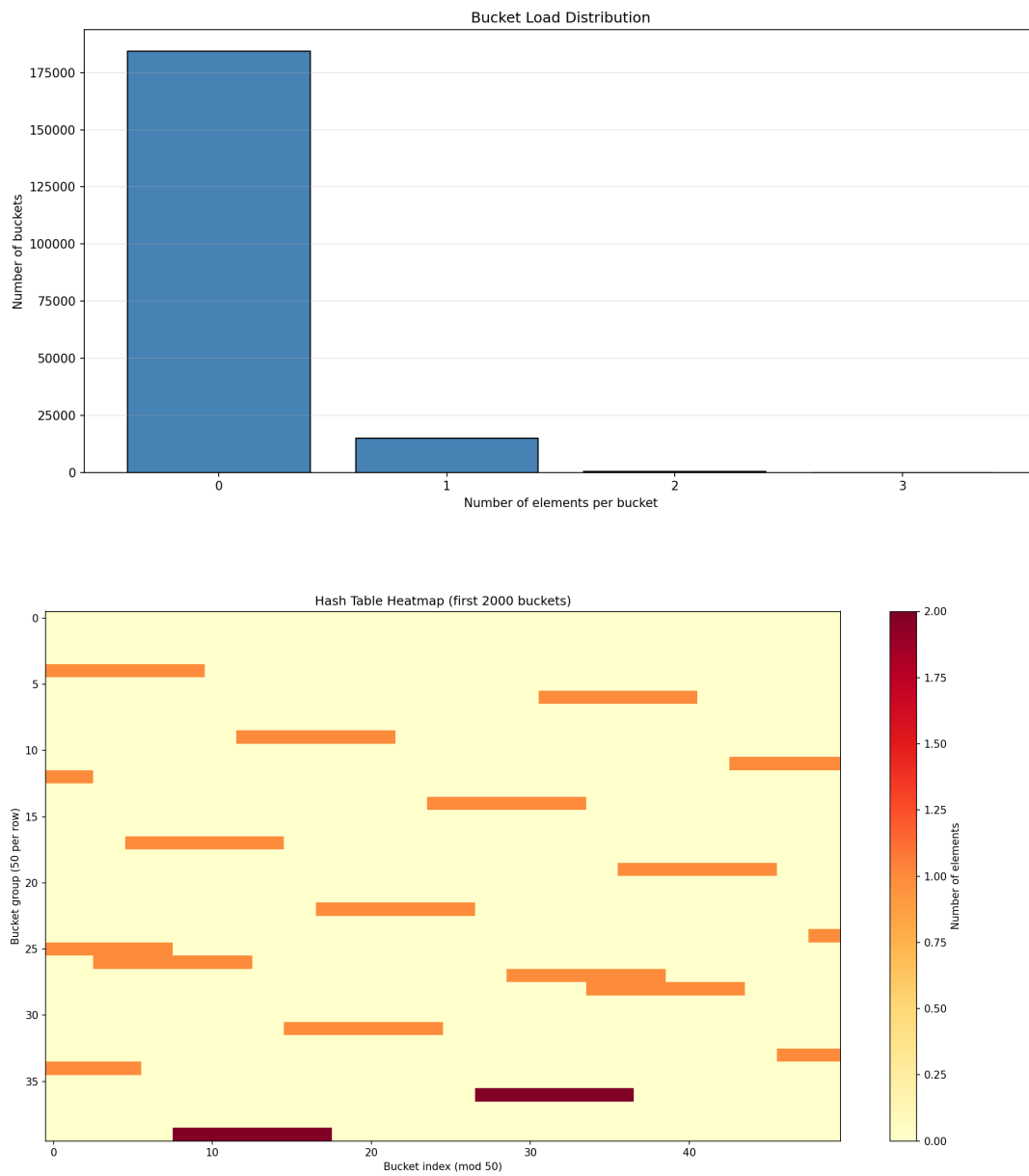


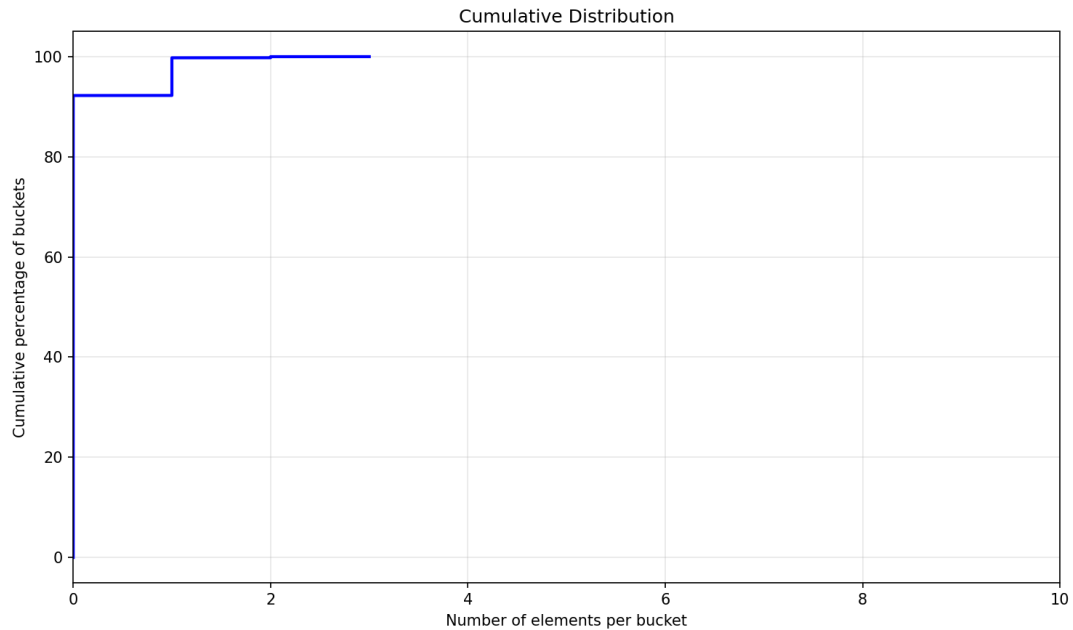
Анализ:

- JS (Sobel): Наименьшее число коллизий (около 300), лучшее время (около 2.5 мкс).
- POLY (полиномиальная): Наша стандартная функция, 475 коллизий, 3.2 мкс хороший компромисс.
- DJB (Bernstein): 450 коллизий, 3.0 мкс близка к POLY.
- RS (Седжвик): Худшая среди тестируемых, более 600 коллизий.

Вывод: Для номеров рейсов (строки вида SU1234) лучше всего подходит JS-функция. Полиномиальная функция с основанием 131 достойная альтернатива.

1.5 Тепловая карта загрузки хэш-таблицы





Анализ:

- Параметры: 500 000 элементов, 2000 уникальных ключей, размер таблицы 200 003.
- Коэффициент загрузки: $2000/200003$ примерно равно 0.01 (1 процент).
- Пустых корзин: около 198 475 (99 процентов таблицы пусто).
- Максимальная загрузка одной корзины: 4 элемента.
- Средняя загрузка непустых корзин: примерно 1.31.
- Тепловая карта показывает равномерное распределение без выраженных кластеров.

Вывод: Хэш-функция распределяет ключи равномерно. Большой размер таблицы (200003) выбран для демонстрации роста коллизий при увеличении числа уникальных ключей. Для реального использования размер таблицы можно уменьшить до 4000-8000 для экономии памяти.

1.6 Общие выводы по лабораторной работе

1. Линейный поиск только для очень малых массивов (менее 1000 элементов).
2. BST и RBT эффективны при группировке дубликатов. При малом числе уникальных ключей работают почти как хэш-таблица. RBT обеспечивает гарантированную логарифмическую высоту.
3. Хэш-таблица лучший выбор для поиска по точному совпадению при большом количестве уникальных ключей (среднее $O(1)$). Размер таблицы должен быть в 2-4 раза больше числа уникальных ключей.
4. `std::multimap` удобен, но уступает самодельным структурам с группировкой дубликатов.
5. Хэш-функция критически важна для производительности. Для строк с номерами рейсов лучшей оказалась JS-функция.
6. Коллизии растут с увеличением числа уникальных ключей при фиксированном размере таблицы. Оптимальный размер таблицы 2-4 умножить на число уникальных ключей.

Глава 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BSTNode		
	Узел бинарного дерева поиска	13
BSTree		
	Класс бинарного дерева поиска	15
DistributionStats		18
Flight		
	Представляет информацию об авиарейсе. Поля: номер рейса (ключ), авиакомпания, дата/время прилёта, число пассажиров	19
FlightRecord		
	Вспомогательная структура для временного хранения записи	21
HashFunctionWrapper		22
HashTable		
	Хэш-таблица для хранения и поиска объектов Flight по ключу	23
HashTableWithHash< HashFunc >		26
RBNode		
	Узел красно-чёрного дерева	28
RBTree		
	Класс красно-чёрного дерева	30

Глава 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

benchmark_hash_functions.cpp	Сравнение различных хэш-функций для строк	35
benchmark_hash_size.cpp	Бенчмарк зависимости коллизий и времени поиска от размера хэш-таблицы . . .	39
benchmark_unique_keys.cpp	Бенчмарк зависимости времени поиска от количества уникальных ключей	44
BSTree.cpp	Реализация методов бинарного дерева поиска	48
BSTree.hpp	Заголовочный файл бинарного дерева поиска (BST)	48
Flight.cpp	Реализация ввода/вывода Flight	49
Flight.hpp	Класс авиарейса (ключ - номер рейса)	50
generate_data.cpp	Генератор тестовых данных для лабораторной работы №2 (поиск данных)	51
hash_heatmap.cpp	Анализ распределения элементов по корзинам хэш-таблицы	55
HashTable.cpp	Реализация хэш-таблицы	57
HashTable.hpp	Заголовочный файл хэш-таблицы с методом цепочек	57
main.cpp	Программа для сравнения времени поиска в различных структурах данных . . .	58
RBTree.cpp	Реализация методов красно-чёрного дерева	60
RBTree.hpp	Заголовочный файл красно-чёрного дерева (Red-Black Tree)	60

Глава 4

Class Documentation

4.1 BSTNode Struct Reference

Узел бинарного дерева поиска.

```
#include <BSTree.hpp>
```

Public Member Functions

- `BSTNode` (const string &k, const `Flight` &f)
Конструктор узла.

Public Attributes

- string `key`
Ключ (номер рейса).
- vector< `Flight` > `values`
Все рейсы с этим ключом
- `BSTNode` * `left`
Указатель на левое поддерево
- `BSTNode` * `right`
Указатель на правое поддерево

4.1.1 Detailed Description

Узел бинарного дерева поиска.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 BSTNode()

```
BSTNode::BSTNode (  
    const string & k,  
    const Flight & f)
```

Конструктор узла.

Parameters

k	Ключ.
f	Объект Flight (добавляется в вектор values).

4.1.3 Member Data Documentation

4.1.3.1 key

```
string BSTNode::key
```

Ключ (номер рейса).

4.1.3.2 left

```
BSTNode\* BSTNode::left
```

Указатель на левое поддерево

4.1.3.3 right

```
BSTNode\* BSTNode::right
```

Указатель на правое поддерево

4.1.3.4 values

```
vector<Flight> BSTNode::values
```

Все рейсы с этим ключом

The documentation for this struct was generated from the following files:

- [BSTree.hpp](#)
- [BSTree.cpp](#)

4.2 BSTree Class Reference

Класс бинарного дерева поиска.

```
#include <BSTree.hpp>
```

Public Member Functions

- [BSTree](#) ()
Конструктор. Инициализирует пустое дерево.
- [~BSTree](#) ()
Деструктор. Освобождает всю память дерева.
- void [insert](#) (const [Flight](#) &f)
Вставка объекта [Flight](#) в дерево.
- vector< [Flight](#) > [find](#) (const string &key) const
Поиск всех объектов с заданным ключом.
- void [clear](#) ()
Очистка дерева (удаление всех узлов).

Private Member Functions

- void [insert](#) (BSTNode *&node, const [Flight](#) &f)
Рекурсивная вставка.
- [BSTNode](#) * [findNode](#) (BSTNode *node, const string &key) const
Рекурсивный поиск узла по ключу.
- void [clear](#) (BSTNode *node)
Рекурсивная очистка поддерева.

Private Attributes

- [BSTNode](#) * [root](#)
Корень дерева

4.2.1 Detailed Description

Класс бинарного дерева поиска.

Обеспечивает:

- Вставку элементов с дубликатами.
- Поиск всех элементов по ключу.
- Очистку дерева.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 BSTree()

BSTree::BSTree ()

Конструктор. Инициализирует пустое дерево.

4.2.2.2 ~BSTree()

BSTree::~~BSTree ()

Деструктор. Освобождает всю память дерева.

4.2.3 Member Function Documentation

4.2.3.1 clear() [1/2]

void BSTree::clear ()

Очистка дерева (удаление всех узлов).

4.2.3.2 clear() [2/2]

void BSTree::clear (
 BSTNode * node) [private]

Рекурсивная очистка поддерева.

4.2.3.3 find()

vector< Flight > BSTree::find (
 const string & key) const

Поиск всех объектов с заданным ключом.

Parameters

key	Номер рейса для поиска.
-----	-------------------------

Returns

Вектор объектов Flight (может быть пустым, если ключ не найден).

4.2.3.4 findNode()

```
BSTNode * BSTree::findNode (  
    BSTNode * node,  
    const string & key) const    [private]
```

Рекурсивный поиск узла по ключу.

4.2.3.5 insert() [1/2]

```
void BSTree::insert (  
    BSTNode *& node,  
    const Flight & f)    [private]
```

Рекурсивная вставка.

4.2.3.6 insert() [2/2]

```
void BSTree::insert (  
    const Flight & f)
```

Вставка объекта [Flight](#) в дерево.

Parameters

f	Объект для вставки.
---	---------------------

Если ключ (flightNumber) уже существует, объект добавляется в вектор values соответствующего узла. Иначе создаётся новый узел.

4.2.4 Member Data Documentation

4.2.4.1 root

```
BSTNode* BSTree::root    [private]
```

Корень дерева

The documentation for this class was generated from the following files:

- [BSTree.hpp](#)
- [BSTree.cpp](#)

4.3 DistributionStats Struct Reference

Public Attributes

- `size_t` [totalBuckets](#)
- `size_t` [nonEmptyBuckets](#)
- `size_t` [maxLoad](#)
- `double` [avgLoad](#)
- `double` [loadFactor](#)
- `size_t` [collisions](#)
- `map< size_t, size_t >` [loadDistribution](#)

4.3.1 Member Data Documentation

4.3.1.1 avgLoad

`double` `DistributionStats::avgLoad`

4.3.1.2 collisions

`size_t` `DistributionStats::collisions`

4.3.1.3 loadDistribution

`map<size_t, size_t>` `DistributionStats::loadDistribution`

4.3.1.4 loadFactor

`double` `DistributionStats::loadFactor`

4.3.1.5 maxLoad

`size_t` `DistributionStats::maxLoad`

4.3.1.6 nonEmptyBuckets

`size_t` `DistributionStats::nonEmptyBuckets`

4.3.1.7 totalBuckets

`size_t` `DistributionStats::totalBuckets`

The documentation for this struct was generated from the following file:

- [hash_heatmap.cpp](#)

4.4 Flight Class Reference

Представляет информацию об авиарейсе. Поля: номер рейса (ключ), авиакомпания, дата/время прилёта, число пассажиров.

```
#include <Flight.hpp>
```

Public Member Functions

- `Flight ()`=default
- `Flight (const string &fn, const string &al, const string &date, const string &time, int pass)`
Конструктор с параметрами.
- `const string &getKey () const`
Возвращает ключ (номер рейса) по константной ссылке (без копирования).

Private Attributes

- `string flightNumber`
Номер рейса (ключ поиска).
- `string airline`
- `string arrivalDate`
- `string arrivalTime`
- `int passengers`

Friends

- `istream & operator>> (istream &is, Flight &f)`
Чтение из CSV-потока (формат: номер,авиа,дата,время,пассажиры).
- `ostream & operator<< (ostream &os, const Flight &f)`
Запись в CSV-поток.

4.4.1 Detailed Description

Представляет информацию об авиарейсе. Поля: номер рейса (ключ), авиакомпания, дата/время прилёта, число пассажиров.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 Flight() [1/2]

`Flight::Flight ()` [default]

4.4.2.2 Flight() [2/2]

```
Flight::Flight (
    const string & fn,
    const string & al,
    const string & date,
    const string & time,
    int pass)
```

Конструктор с параметрами.

Parameters

fn	Номер рейса.
al	Авиакомпания.
date	Дата прилёта.
time	Время прилёта.
pass	Число пассажиров.

4.4.3 Member Function Documentation

4.4.3.1 getKey()

```
const string & Flight::getKey () const [inline]
```

Возвращает ключ (номер рейса) по константной ссылке (без копирования).

4.4.4 Friends And Related Symbol Documentation

4.4.4.1 operator<<

```
ostream & operator<< (
    ostream & os,
    const Flight & f) [friend]
```

Запись в CSV-поток.

4.4.4.2 operator>>

```
istream & operator>> (
    istream & is,
    Flight & f) [friend]
```

Чтение из CSV-потока (формат: номер,авиа,дата,время,пассажиры).

4.4.5 Member Data Documentation

4.4.5.1 airline

```
string Flight::airline [private]
```

4.4.5.2 arrivalDate

string Flight::arrivalDate [private]

4.4.5.3 arrivalTime

string Flight::arrivalTime [private]

4.4.5.4 flightNumber

string Flight::flightNumber [private]

Номер рейса (ключ поиска).

4.4.5.5 passengers

int Flight::passengers [private]

The documentation for this class was generated from the following files:

- [Flight.hpp](#)
- [Flight.cpp](#)

4.5 FlightRecord Struct Reference

Вспомогательная структура для временного хранения записи.

Public Attributes

- std::string [flightNumber](#)
Номер рейса (ключ).
- std::string [airline](#)
Авиакомпания
- std::string [arrivalDate](#)
Дата прилёта
- std::string [arrivalTime](#)
Время прилёта
- int [passengers](#)
Число пассажиров

4.5.1 Detailed Description

Вспомогательная структура для временного хранения записи.

Не является частью основной программы, используется только при генерации CSV-файлов.

4.5.2 Member Data Documentation

4.5.2.1 airline

`std::string FlightRecord::airline`

Авиакомпания

4.5.2.2 arrivalDate

`std::string FlightRecord::arrivalDate`

Дата прилёта

4.5.2.3 arrivalTime

`std::string FlightRecord::arrivalTime`

Время прилёта

4.5.2.4 flightNumber

`std::string FlightRecord::flightNumber`

Номер рейса (ключ).

4.5.2.5 passengers

`int FlightRecord::passengers`

Число пассажиров

The documentation for this struct was generated from the following file:

- [generate_data.cpp](#)

4.6 HashFunctionWrapper Struct Reference

Public Attributes

- string [name](#)
Название функции
- string [description](#)
Краткое описание
- `function< size_t(const string &, size_t)>` [func](#)
Указатель на функцию

4.6.1 Member Data Documentation

4.6.1.1 description

string HashFunctionWrapper::description

Краткое описание

4.6.1.2 func

function<size_t(const string&, size_t)> HashFunctionWrapper::func

Указатель на функцию

4.6.1.3 name

string HashFunctionWrapper::name

Название функции

The documentation for this struct was generated from the following file:

- [benchmark_hash_functions.cpp](#)

4.7 HashTable Class Reference

Хэш-таблица для хранения и поиска объектов [Flight](#) по ключу.

```
#include <HashTable.hpp>
```

Public Member Functions

- [HashTable](#) (size_t table_size=200003)
Конструктор.
- [~HashTable](#) ()
Деструктор.
- void [insert](#) (const [Flight](#) &f)
Вставка объекта в таблицу.
- vector< [Flight](#) > [find](#) (const string &key) const
Поиск всех объектов по ключу.
- size_t [getCollisions](#) () const
- void [clear](#) ()
Очистка таблицы (удаление всех элементов и сброс счётчика коллизий).
- vector< size_t > [getBucketSizes](#) () const

Private Member Functions

- `size_t hash (const string &key) const`
Хэш-функция для строки.

Private Attributes

- `size_t tableSize`
Количество корзин
- `vector< unordered_map< string, vector< Flight > > > buckets`
Корзины
- `size_t collisions`
Счётчик коллизий

4.7.1 Detailed Description

Хэш-таблица для хранения и поиска объектов [Flight](#) по ключу.

4.7.2 Constructor & Destructor Documentation

4.7.2.1 HashTable()

```
HashTable::HashTable (
    size_t table_size = 200003) [explicit]
```

Конструктор.

Parameters

<code>table_size</code>	Размер таблицы (количество корзин).
-------------------------	-------------------------------------

4.7.2.2 ~HashTable()

```
HashTable::~HashTable ()
```

Деструктор.

4.7.3 Member Function Documentation

4.7.3.1 clear()

```
void HashTable::clear ()
```

Очистка таблицы (удаление всех элементов и сброс счётчика коллизий).

4.7.3.2 find()

```
vector< Flight > HashTable::find (  
    const string & key) const
```

Поиск всех объектов по ключу.

Parameters

key	Номер рейса.
-----	--------------

Returns

Вектор копий найденных объектов (пустой, если не найдены).

4.7.3.3 getBucketSizes()

```
vector< size_t > HashTable::getBucketSizes () const [inline]
```

4.7.3.4 getCollisions()

```
size_t HashTable::getCollisions () const [inline]
```

Returns

Общее количество коллизий (конфликтов разных ключей).

4.7.3.5 hash()

```
size_t HashTable::hash (  
    const string & key) const [private]
```

Хэш-функция для строки.

Parameters

key	Входная строка.
-----	-----------------

Returns

Индекс корзины в диапазоне [0, tableSize-1].

4.7.3.6 insert()

```
void HashTable::insert (
    const Flight & f)
```

Вставка объекта в таблицу.

Parameters

f	Объект Flight .
---	---------------------------------

Если ключ (номер рейса) уже существует в соответствующей корзине, объект добавляется в существующий вектор. Если ключ отсутствует, создаётся новая запись. Коллизия увеличивается, если при вставке нового уникального ключа корзина уже была непустьа.

4.7.4 Member Data Documentation

4.7.4.1 buckets

```
vector<unordered_map<string, vector<Flight> > > HashTable::buckets [private]
```

Корзины

4.7.4.2 collisions

```
size_t HashTable::collisions [private]
```

Счётчик коллизий

4.7.4.3 tableSize

```
size_t HashTable::tableSize [private]
```

Количество корзин

The documentation for this class was generated from the following files:

- [HashTable.hpp](#)
- [HashTable.cpp](#)

4.8 HashTableWithHash< HashFunc > Class Template Reference

Public Member Functions

- [HashTableWithHash](#) (size_t table_size, HashFunc hash_func)
- void [insert](#) (const [Flight](#) &f)
- vector< [Flight](#) > [find](#) (const string &key) const
- size_t [getCollisions](#) () const

Private Attributes

- `size_t` `tableSize`
- `vector< unordered_map< string, vector< Flight > > >` `buckets`
- `size_t` `collisions`
- `HashFunc` `hashFunc`

4.8.1 Constructor & Destructor Documentation

4.8.1.1 HashTableWithHash()

```
template<typename HashFunc>
HashTableWithHash< HashFunc >::HashTableWithHash (
    size_t table_size,
    HashFunc hash_func) [inline]
```

4.8.2 Member Function Documentation

4.8.2.1 find()

```
template<typename HashFunc>
vector< Flight > HashTableWithHash< HashFunc >::find (
    const string & key) const [inline]
```

4.8.2.2 getCollisions()

```
template<typename HashFunc>
size_t HashTableWithHash< HashFunc >::getCollisions () const [inline]
```

4.8.2.3 insert()

```
template<typename HashFunc>
void HashTableWithHash< HashFunc >::insert (
    const Flight & f) [inline]
```

4.8.3 Member Data Documentation

4.8.3.1 buckets

```
template<typename HashFunc>
vector<unordered_map<string, vector<Flight> > > HashTableWithHash< HashFunc >::buckets [private]
```

4.8.3.2 collisions

```
template<typename HashFunc>
size_t HashTableWithHash< HashFunc >::collisions [private]
```

4.8.3.3 hashFunc

```
template<typename HashFunc>
HashFunc HashTableWithHash< HashFunc >::hashFunc [private]
```

4.8.3.4 tableSize

```
template<typename HashFunc>
size_t HashTableWithHash< HashFunc >::tableSize [private]
```

The documentation for this class was generated from the following file:

- [benchmark_hash_functions.cpp](#)

4.9 RBNODE Struct Reference

Узел красно-чёрного дерева.

```
#include <RBTree.hpp>
```

Public Member Functions

- [RBNODE](#) (const string &k, const [Flight](#) &f)
Конструктор узла.

Public Attributes

- string [key](#)
Ключ (номер рейса).
- vector< [Flight](#) > [values](#)
Все рейсы с этим ключом
- [Color](#) [color](#)
Цвет узла
- [RBNODE](#) * [left](#)
Указатель на левое поддерево
- [RBNODE](#) * [right](#)
Указатель на правое поддерево
- [RBNODE](#) * [parent](#)
Указатель на родительский узел

4.9.1 Detailed Description

Узел красно-чёрного дерева.

4.9.2 Constructor & Destructor Documentation

4.9.2.1 RBNode()

```
RBNode::RBNode (  
    const string & k,  
    const Flight & f)
```

Конструктор узла.

Parameters

k	Ключ.
f	Объект Flight (добавляется в вектор values).

Новый узел всегда создаётся красным.

4.9.3 Member Data Documentation

4.9.3.1 color

[Color](#) RBNode::color

Цвет узла

4.9.3.2 key

string RBNode::key

Ключ (номер рейса).

4.9.3.3 left

[RBNode*](#) RBNode::left

Указатель на левое поддерево

4.9.3.4 parent

[RBNode*](#) RBNode::parent

Указатель на родительский узел

4.9.3.5 right

[RBNode*](#) RBNode::right

Указатель на правое поддерево

4.9.3.6 values

vector<[Flight](#)> RBNode::values

Все рейсы с этим ключом

The documentation for this struct was generated from the following files:

- [RBTREE.hpp](#)
- [RBTREE.cpp](#)

4.10 RBTREE Class Reference

Класс красно-чёрного дерева.

```
#include <RBTREE.hpp>
```

Public Member Functions

- [RBTREE](#) ()
Конструктор. Инициализирует пустое дерево.
- [~RBTREE](#) ()
Деструктор. Освобождает память дерева.
- void [insert](#) (const [Flight](#) &f)
Вставка объекта [Flight](#) в дерево.
- vector< [Flight](#) > [find](#) (const string &key) const
Поиск всех объектов с заданным ключом.
- void [clear](#) ()
Очистка дерева (удаление всех узлов).

Private Member Functions

- void [rotateLeft](#) ([RBNode](#) *x)
Левый поворот вокруг узла x.
- void [rotateRight](#) ([RBNode](#) *x)
Правый поворот вокруг узла x.
- void [fixInsert](#) ([RBNode](#) *z)
Восстановление свойств красно-чёрного дерева после вставки.
- [RBNode](#) * [findNode](#) ([RBNode](#) *node, const string &key) const
- void [clear](#) ([RBNode](#) *node)

Private Attributes

- [RBNode](#) * [root](#)
Корень дерева

4.10.1 Detailed Description

Класс красно-чёрного дерева.

Обеспечивает логарифмическое время вставки и поиска благодаря балансировке. Поддерживает дубликаты ключей.

4.10.2 Constructor & Destructor Documentation

4.10.2.1 RBTREE()

```
RBTREE::RBTREE ()
```

Конструктор. Инициализирует пустое дерево.

4.10.2.2 ~RBTREE()

```
RBTREE::~~RBTREE ()
```

Деструктор. Освобождает память дерева.

4.10.3 Member Function Documentation

4.10.3.1 clear() [1/2]

```
void RBTREE::clear ()
```

Очистка дерева (удаление всех узлов).

4.10.3.2 clear() [2/2]

```
void RBTREE::clear (  
    RBNODE * node) [private]
```

4.10.3.3 find()

```
vector< FLIGHT > RBTREE::find (  
    const string & key) const
```

Поиск всех объектов с заданным ключом.

Parameters

key	Номер рейса.
-----	--------------

Returns

Вектор объектов [FLIGHT](#) (пустой, если ключ не найден).

4.10.3.4 findNode()

```
RBNode * RBTREE::findNode (  
    RBNode * node,  
    const string & key) const    [private]
```

4.10.3.5 fixInsert()

```
void RBTREE::fixInsert (  
    RBNode * z)    [private]
```

Восстановление свойств красно-чёрного дерева после вставки.

Parameters

z	Вставленный красный узел.
---	---------------------------

Алгоритм:

- Пока родитель z красный, нарушение свойств.
- В зависимости от того, является ли родитель левым или правым потомком деда, и от цвета дяди применяются перекрашивания и/или повороты.
- В конце корень окрашивается в чёрный цвет.

4.10.3.6 insert()

```
void RBTREE::insert (  
    const Flight & f)
```

Вставка объекта [Flight](#) в дерево.

Parameters

f	Объект для вставки.
---	---------------------

Если ключ уже существует, объект добавляется в вектор существующего узла. Иначе создаётся новый красный узел, после чего выполняются повороты и перекрашивания для восстановления свойств красно-чёрного дерева.

4.10.3.7 rotateLeft()

```
void RBTREE::rotateLeft (  
    RBNode * x) [private]
```

Левый поворот вокруг узла x.

Parameters

x	Узел, вокруг которого выполняется поворот.
---	--

4.10.3.8 rotateRight()

```
void RBTREE::rotateRight (  
    RBNode * x) [private]
```

Правый поворот вокруг узла x.

Parameters

x	Узел, вокруг которого выполняется поворот.
---	--

4.10.4 Member Data Documentation

4.10.4.1 root

```
RBNode* RBTREE::root [private]
```

Корень дерева

The documentation for this class was generated from the following files:

- [RBTREE.hpp](#)
- [RBTREE.cpp](#)

Глава 5

File Documentation

5.1 benchmark_hash_functions.cpp File Reference

Сравнение различных хэш-функций для строк.

```
#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <fstream>
#include <random>
#include <set>
#include <functional>
#include <iomanip>
#include "Flight.hpp"
#include "HashTable.hpp"
```

Classes

- struct [HashFunctionWrapper](#)
- class [HashTableWithHash< HashFunc >](#)

Functions

- size_t [polyHash](#) (const string &key, size_t mod)
Полиномиальная хэш-функция (используется в [HashTable](#) по умолчанию).
- size_t [rsHash](#) (const string &key, size_t mod)
RS хэш-функция (из книги Роберта Седжвика).
- size_t [jsHash](#) (const string &key, size_t mod)
JS хэш-функция (Justin Sobel).
- size_t [djbHash](#) (const string &key, size_t mod)
DJB хэш-функция (Daniel J. Bernstein).
- vector< [HashFunctionWrapper](#) > [getHashFunctions](#) ()
- vector< [Flight](#) > [generateFlights](#) (size_t totalSize, size_t uniqueKeysCount)
- vector< string > [getTestKeys](#) (size_t uniqueKeysCount)
- int [main](#) ()

Variables

- `const size_t TOTAL_SIZE = 500000`
Фиксированный размер массива
- `const size_t UNIQUE_KEYS_COUNT = 2000`
Количество уникальных ключей
- `const size_t HASH_TABLE_SIZE = 200003`
Фиксированный размер таблицы
- `const int NUM_SEARCHES = 100`
Количество поисков для замера
- `const unsigned int RANDOM_SEED = 42`
Фиксированный seed для воспроизводимости

5.1.1 Detailed Description

Сравнение различных хэш-функций для строк.

Сравниваются 4 хэш-функции: POLY (наша текущая), RS, JS, DJB. ROT13 исключён из-за аномально высокого количества коллизий (~15000), что делает его непригодным для хэш-таблиц.

Author

Починова Алина

Date

2026

5.1.2 Function Documentation

5.1.2.1 djbHash()

```
size_t djbHash (
    const string & key,
    size_t mod)
```

DJB хэш-функция (Daniel J. Bernstein).

Parameters

key	Входная строка.
mod	Размер таблицы.

Returns

Хэш-код в диапазоне $[0, \text{mod}-1]$.

5.1.2.2 generateFlights()

```
vector< Flight > generateFlights (
    size_t totalSize,
    size_t uniqueKeysCount)
```

5.1.2.3 getHashFunctions()

```
vector< HashFunctionWrapper > getHashFunctions ()
```

5.1.2.4 getTestKeys()

```
vector< string > getTestKeys (
    size_t uniqueKeysCount)
```

5.1.2.5 jsHash()

```
size_t jsHash (
    const string & key,
    size_t mod)
```

JS хэш-функция (Justin Sobel).

Parameters

key	Входная строка.
mod	Размер таблицы.

Returns

Хэш-код в диапазоне $[0, \text{mod}-1]$.

5.1.2.6 main()

```
int main ()
```

5.1.2.7 polyHash()

```
size_t polyHash (  
    const string & key,  
    size_t mod)
```

Полиномиальная хэш-функция (используется в [HashTable](#) по умолчанию).

Parameters

key	Входная строка.
mod	Размер таблицы.

Returns

Хэш-код в диапазоне [0, mod-1].

5.1.2.8 rsHash()

```
size_t rsHash (  
    const string & key,  
    size_t mod)
```

RS хэш-функция (из книги Роберта Седжвика).

Parameters

key	Входная строка.
mod	Размер таблицы.

Returns

Хэш-код в диапазоне [0, mod-1].

5.1.3 Variable Documentation

5.1.3.1 HASH_TABLE_SIZE

```
const size_t HASH_TABLE_SIZE = 200003
```

Фиксированный размер таблицы

5.1.3.2 NUM_SEARCHES

```
const int NUM_SEARCHES = 100
```

Количество поисков для замера

5.1.3.3 RANDOM_SEED

```
const unsigned int RANDOM_SEED = 42
```

Фиксированный seed для воспроизводимости

5.1.3.4 TOTAL_SIZE

```
const size_t TOTAL_SIZE = 500000
```

Фиксированный размер массива

5.1.3.5 UNIQUE_KEYS_COUNT

```
const size_t UNIQUE_KEYS_COUNT = 2000
```

Количество уникальных ключей

5.2 benchmark_hash_size.cpp File Reference

Бенчмарк зависимости коллизий и времени поиска от размера хэш-таблицы.

```
#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <fstream>
#include <random>
#include <set>
#include <cmath>
#include "Flight.hpp"
#include "HashTable.hpp"
```

Functions

- `bool isPrime (size_t n)`
Проверка, является ли число простым.
- `vector< size_t > generateTableSizes ()`
Генерирует список простых чисел для размеров хэш-таблицы.
- `vector< Flight > generateFlights (size_t totalSize, size_t uniqueKeysCount)`
Генерирует массив объектов `Flight` с фиксированным числом уникальных ключей.
- `vector< string > getTestKeys (size_t uniqueKeysCount)`
Выбирает фиксированные ключи для поиска (первые `NUM_SEARCHES` номеров).
- `double measureSearchTime (const HashTable &ht, const vector< string > &keys)`
Замеряет время поиска в хэш-таблице.
- `int main ()`
Главная функция бенчмарка.

Variables

- `const size_t TOTAL_SIZE = 500000`
Фиксированный размер массива
- `const size_t UNIQUE_KEYS_COUNT = 2000`
Количество уникальных ключей
- `const int NUM_SEARCHES = 100`
Количество поисков для замера
- `const unsigned int RANDOM_SEED = 42`
Фиксированный seed для воспроизводимости

5.2.1 Detailed Description

Бенчмарк зависимости коллизий и времени поиска от размера хэш-таблицы.

Фиксируется массив из 500 000 элементов (2000 уникальных ключей). Размер хэш-таблицы варьируется от 101 до 500009 (простые числа). Для каждого размера измеряется количество коллизий и время поиска.

Author

Починова Алина

Date

2026

5.2.2 Запуск

`./benchmark_hash_size`

5.2.3 Выходные данные

Файл `hash_size_results.csv` с колонками:

- `table_size`: размер хэш-таблицы
- `collisions`: количество коллизий
- `search_time_us`: время поиска (микросекунды)
- `load_factor`: коэффициент заполнения (уникальные_ключи / `table_size`)

5.2.4 Function Documentation

5.2.4.1 generateFlights()

```
vector< Flight > generateFlights (  
    size_t totalSize,  
    size_t uniqueKeysCount)
```

Генерирует массив объектов `Flight` с фиксированным числом уникальных ключей.

Parameters

totalSize	Общее количество элементов.
uniqueKeysCount	Количество различных номеров рейсов.

Returns

Вектор объектов `Flight`.

5.2.4.2 generateTableSizes()

```
vector< size_t > generateTableSizes ()
```

Генерирует список простых чисел для размеров хэш-таблицы.

Returns

Вектор простых чисел в диапазоне от 100 до 600000.

5.2.4.3 getTestKeys()

```
vector< string > getTestKeys (  
    size_t uniqueKeysCount)
```

Выбирает фиксированные ключи для поиска (первые `NUM_SEARCHES` номеров).

Parameters

uniqueKeysCount	Общее количество уникальных ключей.
-----------------	-------------------------------------

Returns

Вектор ключей.

5.2.4.4 isPrime()

```
bool isPrime (  
    size_t n)
```

Проверка, является ли число простым.

Parameters

n	Проверяемое число.
---	--------------------

Returns

true, если число простое.

5.2.4.5 main()

```
int main ()
```

Главная функция бенчмарка.

Returns

0 при успешном выполнении.

Последовательно тестирует хэш-таблицы разных размеров:

- Генерирует фиксированный массив данных (один раз).
- Для каждого размера таблицы строит новую хэш-таблицу, вставляет все элементы, замеряет коллизии и время поиска.
- Сохраняет результаты в CSV.

Note

Ожидаемый результат:

- При малом размере таблицы (101) – много коллизий, время поиска большое.
- При оптимальном размере ($\sim 2 * \text{UNIQUE_KEYS_COUNT} = 4000$) – коллизий мало.
- При дальнейшем увеличении размера – коллизии стремятся к 0, время поиска стабилизируется, но растёт расход памяти.

5.2.4.6 measureSearchTime()

```
double measureSearchTime (  
    const HashTable & ht,  
    const vector< string > & keys)
```

Замеряет время поиска в хэш-таблице.

Parameters

ht	Хэш-таблица.
keys	Вектор ключей для поиска.

Returns

Среднее время одного поиска в микросекундах.

5.2.5 Variable Documentation

5.2.5.1 NUM_SEARCHES

```
const int NUM_SEARCHES = 100
```

Количество поисков для замера

5.2.5.2 RANDOM_SEED

```
const unsigned int RANDOM_SEED = 42
```

Фиксированный seed для воспроизводимости

5.2.5.3 TOTAL_SIZE

```
const size_t TOTAL_SIZE = 500000
```

Фиксированный размер массива

5.2.5.4 UNIQUE_KEYS_COUNT

```
const size_t UNIQUE_KEYS_COUNT = 2000
```

Количество уникальных ключей

5.3 benchmark_unique_keys.cpp File Reference

Бенчмарк зависимости времени поиска от количества уникальных ключей.

```
#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <fstream>
#include <random>
#include <set>
#include <algorithm>
#include "Flight.hpp"
#include "BSTree.hpp"
#include "RBTre.hpp"
#include "HashTable.hpp"
```

Functions

- vector< [Flight](#) > [generateFlights](#) (size_t totalSize, size_t uniqueKeysCount)
Генерирует массив объектов [Flight](#) с заданным количеством уникальных номеров рейсов.
- vector< string > [selectTestKeys](#) (const vector< [Flight](#) > &flights)
Выбирает NUM_SEARCHES случайных ключей из массива для последующего поиска.
- template<typename T>
double [measureSearchTime](#) (const T &structure, const vector< string > &keys)
Замеряет среднее время поиска в структуре данных.
- int [main](#) ()
Главная функция бенчмарка.

Variables

- const size_t [TOTAL_SIZE](#) = 1000000
Фиксированный размер массива
- const size_t [HASH_TABLE_SIZE](#) = 200003
Фиксированный размер хэш-таблицы
- const int [NUM_SEARCHES](#) = 10
Количество поисков для замера
- const unsigned int [RANDOM_SEED](#) = 42
Фиксированный seed для воспроизводимости

5.3.1 Detailed Description

Бенчмарк зависимости времени поиска от количества уникальных ключей.

Фиксируется размер массива (1 000 000 элементов), но меняется количество различных номеров рейсов (от 100 до 500 000). Для каждого значения измеряется время поиска в BST, RBT и [HashTable](#). Ожидаемое поведение: время поиска в BST/RBT должно расти как $O(\log U)$, в хэш-таблице – как $O(1)$ с медленным ростом из-за коллизий. Если наблюдаются немонотонные колебания или падение времени, это может быть вызвано:

1. Эффектами кэширования процессора (данные малых U помещаются в кэш).
2. Шумом от фоновых процессов ОС.
3. Недостаточным количеством поисков (`NUM_SEARCHES`) для усреднения.
4. Случайным выбором ключей (разные ключи могут иметь разную глубину).

Warning

@recommendation Для получения гладкого графика:

- Увеличить `NUM_SEARCHES` до 100.
- Запустить бенчмарк несколько раз и усреднить результаты.
- Использовать одни и те же ключи для всех U (например, первые 10 уникальных).
- Отключить турбобуст процессора и закрыть фоновые приложения.

Author

Починова Алина

Date

2026

5.3.2 Запуск

`./benchmark_unique_keys`

5.3.3 Выходные данные

Файл `unique_keys_results.csv` с колонками:

- `unique_keys`: количество уникальных ключей
- `bst_time_us`: время поиска в BST (микросекунды)
- `rbt_time_us`: время поиска в RBT (микросекунды)
- `hash_time_us`: время поиска в хэш-таблице (микросекунды)

5.3.4 Function Documentation

5.3.4.1 generateFlights()

```
vector< Flight > generateFlights (
    size_t totalSize,
    size_t uniqueKeysCount)
```

Генерирует массив объектов [Flight](#) с заданным количеством уникальных номеров рейсов.

Parameters

totalSize	Общее количество элементов.
uniqueKeysCount	Количество различных номеров рейсов.

Returns

Вектор объектов [Flight](#).

Номера рейсов генерируются в диапазоне [1, uniqueKeysCount]. Используется фиксированный seed для воспроизводимости.

5.3.4.2 main()

```
int main ()
```

Главная функция бенчмарка.

Returns

0 при успешном выполнении.

5.3.4.3 measureSearchTime()

```
template<typename T>
double measureSearchTime (
    const T & structure,
    const vector< string > & keys)
```

Замеряет среднее время поиска в структуре данных.

Template Parameters

Structure	Тип структуры (должен иметь метод find(const string&) const).
-----------	---

Parameters

structure	Структура для поиска.
keys	Вектор ключей для поиска.

Returns

Среднее время одного поиска в микросекундах.

5.3.4.4 selectTestKeys()

```
vector< string > selectTestKeys (  
    const vector< Flight > & flights)
```

Выбирает NUM_SEARCHES случайных ключей из массива для последующего поиска.

Parameters

flights	Массив объектов.
---------	------------------

Returns

Вектор уникальных ключей.

5.3.5 Variable Documentation

5.3.5.1 HASH_TABLE_SIZE

```
const size_t HASH_TABLE_SIZE = 200003
```

Фиксированный размер хэш-таблицы

5.3.5.2 NUM_SEARCHES

```
const int NUM_SEARCHES = 10
```

Количество поисков для замера

5.3.5.3 RANDOM_SEED

```
const unsigned int RANDOM_SEED = 42
```

Фиксированный seed для воспроизводимости

5.3.5.4 TOTAL_SIZE

```
const size_t TOTAL_SIZE = 1000000
```

Фиксированный размер массива

5.4 BSTree.cpp File Reference

Реализация методов бинарного дерева поиска.

```
#include "BSTree.hpp"
```

5.4.1 Detailed Description

Реализация методов бинарного дерева поиска.

5.5 BSTree.hpp File Reference

Заголовочный файл бинарного дерева поиска (BST).

```
#include <vector>
#include <string>
#include "Flight.hpp"
```

Classes

- struct [BSTNode](#)
Узел бинарного дерева поиска.
- class [BSTree](#)
Класс бинарного дерева поиска.

5.5.1 Detailed Description

Заголовочный файл бинарного дерева поиска (BST).

Реализует бинарное дерево поиска для хранения объектов [Flight](#). Ключом является номер рейса (flightNumber). Поддерживаются дубликаты: в каждом узле хранится вектор всех объектов с одинаковым ключом.

5.6 BSTree.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #pragma once
00011 #include <vector>
00012 #include <string>
00013 #include "Flight.hpp"
00014
00015 using namespace std;
00016
00021 struct BSTNode {
00022     string key;
00023     vector<Flight> values;
00024     BSTNode* left;
00025     BSTNode* right;
00026
00032     BSTNode(const string& k, const Flight& f);
00033 };
00034
00044 class BSTree {
00045 public:
00047     BSTree();
00048
00050     ~BSTree();
00051
00058     void insert(const Flight& f);
00059
00065     vector<Flight> find(const string& key) const;
00066
00068     void clear();
00069
00070 private:
00071     BSTNode* root;
00072
00074     void insert(BSTNode*& node, const Flight& f);
00075
00077     BSTNode* findNode(BSTNode* node, const string& key) const;
00078
00080     void clear(BSTNode* node);
00081 };

```

5.7 Flight.cpp File Reference

Реализация ввода/вывода `Flight`.

```

#include "Flight.hpp"
#include <sstream>

```

Functions

- `istream & operator>>` (`istream &is`, `Flight &f`)
- `ostream & operator<<` (`ostream &os`, `const Flight &f`)

5.7.1 Detailed Description

Реализация ввода/вывода `Flight`.

5.7.2 Function Documentation

5.7.2.1 `operator<<()`

```

ostream & operator<< (
    ostream & os,
    const Flight & f)

```

5.7.2.2 operator>>()

```
istream & operator>> (
    istream & is,
    Flight & f)
```

5.8 Flight.hpp File Reference

Класс авиарейса (ключ - номер рейса).

```
#include <string>
#include <iostream>
```

Classes

- class [Flight](#)

Представляет информацию об авиарейсе. Поля: номер рейса (ключ), авиакомпания, дата/время прилёта, число пассажиров.

5.8.1 Detailed Description

Класс авиарейса (ключ - номер рейса).

5.9 Flight.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007 #include <string>
00008 #include <iostream>
00009
00010 using namespace std;
00011
00017 class Flight {
00018 private:
00019     string flightNumber;
00020     string airline;
00021     string arrivalDate;
00022     string arrivalTime;
00023     int passengers;
00024
00025 public:
00026     Flight() = default;
00027
00036     Flight(const string& fn, const string& al,
00037           const string& date, const string& time, int pass);
00038
00040     const string& getKey() const { return flightNumber; }
00041
00043     friend istream& operator»(istream& is, Flight& f);
00044
00046     friend ostream& operator«(ostream& os, const Flight& f);
00047 };
```

5.10 generate_data.cpp File Reference

Генератор тестовых данных для лабораторной работы №2 (поиск данных).

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <random>
#include <iomanip>
#include <sstream>
#include <sys/stat.h>
```

Classes

- struct [FlightRecord](#)
Вспомогательная структура для временного хранения записи.

Functions

- `std::mt19937 gen (RANDOM_SEED)`
Глобальный генератор случайных чисел (Mersenne Twister).
- `std::string randomDate ()`
Генерация случайной даты прилёта.
- `std::string randomTime ()`
Генерация случайного времени прилёта.
- `std::string randomAirline ()`
Генерация случайного названия авиакомпании.
- `int randomPassengers ()`
Генерация случайного числа пассажиров.
- `std::string randomFlightNumber ()`
Генерация номера рейса из ограниченного множества.
- `void saveToCSV (const std::vector< FlightRecord > &flights, const std::string &filename)`
Сохранение вектора записей в CSV-файл.
- `int main ()`
Главная функция генератора данных.

Variables

- `const int UNIQUE_FLIGHT_NUMBERS = 2000`
Количество уникальных номеров рейсов.
- `const unsigned int RANDOM_SEED = 42`
Фиксированный seed для генератора (воспроизводимость).
- `const std::vector< std::string > AIRLINES`
Список авиакомпаний для случайного выбора.
- `const std::vector< std::string > PREFIXES = {"SU", "S7", "FV", "DP", "U6", "UT", "N4", "WZ"}`
Префиксы для номеров рейсов.

5.10.1 Detailed Description

Генератор тестовых данных для лабораторной работы №2 (поиск данных).

Создаёт CSV-файлы с информацией об авиарейсах. Номера рейсов (ключи поиска) генерируются из ограниченного набора, что обеспечивает многократные повторения для тестирования поиска всех вхождений.

5.10.2 Запуск

```
./generate_data
```

5.10.3 Выходные данные

Файлы сохраняются в папку data/ с именами flights_{size}.csv, где size – количество записей.

See also

[Flight](#) (класс из основной программы)
main_benchmark.cpp

5.10.4 Function Documentation

5.10.4.1 gen()

```
std::mt19937 gen (
    RANDOM\_SEED )
```

Глобальный генератор случайных чисел (Mersenne Twister).

Инициализируется фиксированным seed для воспроизводимости.

5.10.4.2 main()

```
int main ()
```

Главная функция генератора данных.

Returns

0 при успешном завершении.

Последовательно генерирует файлы для следующих размеров: 100, 500, 1000, 5000, 10000, 20000, 30000, 50000, 75000, 100000, 200000, 300000, 500000, 750000, 1000000.

Postcondition

В папке data/ создаются файлы flights_{size}.csv.

5.10.4.3 randomAirline()

`std::string randomAirline ()`

Генерация случайного названия авиакомпании.

Returns

Одна из строк в списке AIRLINES.

5.10.4.4 randomDate()

`std::string randomDate ()`

Генерация случайной даты прилёта.

Returns

Строка в формате YYYY-MM-DD.

Диапазон: 2024-01-01 до 2025-12-28.

5.10.4.5 randomFlightNumber()

`std::string randomFlightNumber ()`

Генерация номера рейса из ограниченного множества.

Returns

Строка вида "SU1234", где число от 1 до UNIQUE_FLIGHT_NUMBERS.

Номер рейса формируется как случайный префикс + число. Такая схема гарантирует, что количество различных ключей не превышает `UNIQUE_FLIGHT_NUMBERS * PREFIXES.size()`, что создаёт множество дубликатов в больших массивах.

5.10.4.6 randomPassengers()

`int randomPassengers ()`

Генерация случайного числа пассажиров.

Returns

Число от 0 до 300.

5.10.4.7 randomTime()

```
std::string randomTime ()
```

Генерация случайного времени прилёта.

Returns

Строка в формате HH:MM:SS.

Диапазон: 00:00:00 до 23:59:59.

5.10.4.8 saveToCSV()

```
void saveToCSV (
    const std::vector< FlightRecord > & flights,
    const std::string & filename)
```

Сохранение вектора записей в CSV-файл.

Parameters

flights	Вектор структур FlightRecord .
filename	Имя выходного файла.

Заголовок файла соответствует ожиданиям класса [Flight](#) из основной программы.

5.10.5 Variable Documentation

5.10.5.1 AIRLINES

```
const std::vector<std::string> AIRLINES
```

Initial value:

```
= {
    "Aeroflot", "S7 Airlines", "Rossiya", "Pobeda",
    "Ural Airlines", "Utair", "Nordwind", "Red Wings"
}
```

Список авиакомпаний для случайного выбора.

5.10.5.2 PREFIXES

```
const std::vector<std::string> PREFIXES = {"SU", "S7", "FV", "DP", "U6", "UT", "N4", "WZ"}
```

Префиксы для номеров рейсов.

5.10.5.3 RANDOM_SEED

```
const unsigned int RANDOM_SEED = 42
```

Фиксированный seed для генератора (воспроизводимость).

5.10.5.4 UNIQUE_FLIGHT_NUMBERS

```
const int UNIQUE_FLIGHT_NUMBERS = 2000
```

Количество уникальных номеров рейсов.

Чем меньше это число, тем больше дубликатов в массиве. При размере N каждый ключ в среднем встретится $N / \text{UNIQUE_COUNT}$ раз.

5.11 hash_heatmap.cpp File Reference

Анализ распределения элементов по корзинам хэш-таблицы.

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <iomanip>
#include <algorithm>
#include <random>
#include <map>
#include "Flight.hpp"
#include "HashTable.hpp"
```

Classes

- struct [DistributionStats](#)

Functions

- vector< [Flight](#) > [generateFlights](#) (size_t totalSize, size_t uniqueKeysCount)
- [DistributionStats](#) [analyzeDistribution](#) (const [HashTable](#) &ht)
- void [saveTextReport](#) (const [DistributionStats](#) &stats, const string &filename)
- void [saveBucketLoadCSV](#) (const [HashTable](#) &ht, const string &filename)
- int [main](#) ()

Variables

- const size_t [TOTAL_SIZE](#) = 500000
- const size_t [UNIQUE_KEYS_COUNT](#) = 2000
- const size_t [HASH_TABLE_SIZE](#) = 200003
- const unsigned int [RANDOM_SEED](#) = 42

5.11.1 Detailed Description

Анализ распределения элементов по корзинам хэш-таблицы.

Строит хэш-таблицу на 500000 элементах (2000 уникальных ключей), анализирует распределение элементов по корзинам, выводит статистику. Для визуализации используется Python-скрипт `plot_heatmap.py`.

5.11.2 Выходные данные

- hash_distribution.txt: текстовый отчёт со статистикой
- bucket_load.csv: данные о загрузке корзин

5.11.3 Для построения тепловой карты:

```
python3 plot_heatmap.py
```

5.11.4 Function Documentation

5.11.4.1 analyzeDistribution()

```
DistributionStats analyzeDistribution (  
    const HashTable & ht)
```

5.11.4.2 generateFlights()

```
vector< Flight > generateFlights (  
    size_t totalSize,  
    size_t uniqueKeysCount)
```

5.11.4.3 main()

```
int main ()
```

5.11.4.4 saveBucketLoadCSV()

```
void saveBucketLoadCSV (  
    const HashTable & ht,  
    const string & filename)
```

5.11.4.5 saveTextReport()

```
void saveTextReport (  
    const DistributionStats & stats,  
    const string & filename)
```

5.11.5 Variable Documentation

5.11.5.1 HASH_TABLE_SIZE

```
const size_t HASH_TABLE_SIZE = 200003
```


5.11.5.2 RANDOM_SEED

```
const unsigned int RANDOM_SEED = 42
```

5.11.5.3 TOTAL_SIZE

```
const size_t TOTAL_SIZE = 500000
```

5.11.5.4 UNIQUE_KEYS_COUNT

```
const size_t UNIQUE_KEYS_COUNT = 2000
```

5.12 HashTable.cpp File Reference

Реализация хэш-таблицы.

```
#include "HashTable.hpp"
```

5.12.1 Detailed Description

Реализация хэш-таблицы.

5.13 HashTable.hpp File Reference

Заголовочный файл хэш-таблицы с методом цепочек.

```
#include <vector>
#include <unordered_map>
#include <string>
#include "Flight.hpp"
```

Classes

- class [HashTable](#)
Хэш-таблица для хранения и поиска объектов [Flight](#) по ключу.

5.13.1 Detailed Description

Заголовочный файл хэш-таблицы с методом цепочек.

Реализует хэш-таблицу, где каждая корзина является `unordered_map<key, vector<Flight>>`. Это позволяет хранить дубликаты и корректно подсчитывать коллизии (только для разных ключей, попавших в одну корзину).

5.14 HashTable.hpp

[Go to the documentation of this file.](#)

```

00001
00010
00011 #pragma once
00012 #include <vector>
00013 #include <unordered_map>
00014 #include <string>
00015 #include "Flight.hpp"
00016
00017 using namespace std;
00018
00023 class HashTable {
00024 public:
00029     explicit HashTable(size_t table_size = 200003);
00030
00032     ~HashTable();
00033
00043     void insert(const Flight& f);
00044
00050     vector<Flight> find(const string& key) const;
00051
00053     size_t getCollisions() const { return collisions; }
00054
00056     void clear();
00057
00058     vector<size_t> getBucketSizes() const {
00059         vector<size_t> sizes;
00060         sizes.reserve(buckets.size());
00061         for (const auto& bucket : buckets) {
00062             sizes.push_back(bucket.size());
00063         }
00064         return sizes;
00065     }
00066
00067 private:
00073     size_t hash(const string& key) const;
00074
00075     size_t tableSize;
00076     vector<unordered_map<string, vector<Flight>>> buckets;
00077     size_t collisions;
00078 };

```

5.15 main.cpp File Reference

Программа для сравнения времени поиска в различных структурах данных.

```

#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <chrono>
#include <map>
#include <algorithm>
#include <set>
#include "Flight.hpp"
#include "BSTree.hpp"
#include "RBTREE.hpp"
#include "HashTable.hpp"

```

Functions

- vector< [Flight](#) > [loadFromCSV](#) (const string &filename)
Загрузка данных из CSV-файла.
- vector< [Flight](#) > [linearSearch](#) (const vector< [Flight](#) > &data, const string &key)

Линейный поиск всех вхождений ключа в массиве.

- `vector< string > getUniqueKeys (const vector< Flight > &data, size_t count)`

Получение первых N уникальных ключей из массива.

- `int main ()`

5.15.1 Detailed Description

Программа для сравнения времени поиска в различных структурах данных.

Загружает данные об авиарейсах из CSV-файлов разного размера, строит структуры поиска (BST, красно-чёрное дерево, хэш-таблица, `std::multimap`) и замеряет время выполнения 10 поисковых запросов. Результаты сохраняются в CSV-файл для последующего построения графиков.

See also

[Flight](#), [BSTree](#), [RBTree](#), [HashTable](#)

5.15.2 Function Documentation

5.15.2.1 `getUniqueKeys()`

```
vector< string > getUniqueKeys (  
    const vector< Flight > & data,  
    size_t count)
```

Получение первых N уникальных ключей из массива.

Parameters

data	Вектор объектов.
count	Желаемое количество уникальных ключей.

Returns

Вектор уникальных ключей (не более count).

5.15.2.2 `linearSearch()`

```
vector< Flight > linearSearch (  
    const vector< Flight > & data,  
    const string & key)
```

Линейный поиск всех вхождений ключа в массиве.

Parameters

data	Вектор объектов.
------	------------------

key	Искомый ключ (номер рейса).
-----	-----------------------------

Returns

Вектор найденных объектов.

5.15.2.3 loadFromCSV()

```
vector< Flight > loadFromCSV (  
    const string & filename)
```

Загрузка данных из CSV-файла.

Parameters

filename	Имя файла.
----------	------------

Returns

Вектор объектов [Flight](#).

5.15.2.4 main()

```
int main ()
```

5.16 RBTTree.cpp File Reference

Реализация методов красно-чёрного дерева.

```
#include "RBTTree.hpp"
```

5.16.1 Detailed Description

Реализация методов красно-чёрного дерева.

5.17 RBTTree.hpp File Reference

Заголовочный файл красно-чёрного дерева (Red-Black Tree).

```
#include <vector>  
#include <string>  
#include "Flight.hpp"
```

Classes

- struct [RBNode](#)
Узел красно-чёрного дерева.
- class [RBTree](#)
Класс красно-чёрного дерева.

Enumerations

- enum [Color](#) { [RED](#) , [BLACK](#) }
Цвет узла красно-чёрного дерева.

5.17.1 Detailed Description

Заголовочный файл красно-чёрного дерева (Red-Black Tree).

Реализует самобалансирующееся двоичное дерево поиска с поддержкой дубликатов. Каждый узел хранит ключ (номер рейса) и вектор объектов [Flight](#) с этим ключом.

5.17.2 Enumeration Type Documentation

5.17.2.1 Color

enum [Color](#)

Цвет узла красно-чёрного дерева.

Enumerator

RED	
BLACK	

5.18 RBTREE.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #pragma once
00011 #include <vector>
00012 #include <string>
00013 #include "Flight.hpp"
00014
00015 using namespace std;
00016
00021 enum Color { RED, BLACK };
00022
00027 struct RBNode {
00028     string key;
00029     vector<Flight> values;
00030     Color color;
00031     RBNode* left;
00032     RBNode* right;
00033     RBNode* parent;
00034
00041     RBNode(const string& k, const Flight& f);
00042 };
00043

```

```
00051 class RBTREE {
00052 public:
00054     RBTREE();
00055
00057     ~RBTREE();
00058
00067     void insert(const Flight& f);
00068
00074     vector<Flight> find(const string& key) const;
00075
00077     void clear();
00078
00079 private:
00080     RBNode* root;
00081     void rotateLeft(RBNode* x);
00082     void rotateRight(RBNode* x);
00083     void fixInsert(RBNode* z);
00084     RBNode* findNode(RBNode* node, const string& key) const;
00085     void clear(RBNode* node);
00086 };
```

Предметный указатель

- ~BSTree
 - BSTree, [16](#)
- ~HashTable
 - HashTable, [24](#)
- ~RBTre
 - RBTre, [31](#)
- airline
 - Flight, [20](#)
 - FlightRecord, [22](#)
- AIRLINES
 - generate_data.cpp, [54](#)
- analyzeDistribution
 - hash_heatmap.cpp, [56](#)
- arrivalDate
 - Flight, [20](#)
 - FlightRecord, [22](#)
- arrivalTime
 - Flight, [21](#)
 - FlightRecord, [22](#)
- avgLoad
 - DistributionStats, [18](#)
- benchmark_hash_functions.cpp, [35](#)
 - djbHash, [36](#)
 - generateFlights, [36](#)
 - getHashFunctions, [37](#)
 - getTestKeys, [37](#)
 - HASH_TABLE_SIZE, [38](#)
 - jsHash, [37](#)
 - main, [37](#)
 - NUM_SEARCHES, [38](#)
 - polyHash, [37](#)
 - RANDOM_SEED, [38](#)
 - rsHash, [38](#)
 - TOTAL_SIZE, [39](#)
 - UNIQUE_KEYS_COUNT, [39](#)
- benchmark_hash_size.cpp, [39](#)
 - generateFlights, [41](#)
 - generateTableSizes, [41](#)
 - getTestKeys, [41](#)
 - isPrime, [41](#)
 - main, [42](#)
 - measureSearchTime, [42](#)
 - NUM_SEARCHES, [43](#)
 - RANDOM_SEED, [43](#)
 - TOTAL_SIZE, [43](#)
 - UNIQUE_KEYS_COUNT, [43](#)
- benchmark_unique_keys.cpp, [44](#)
 - generateFlights, [46](#)
 - HASH_TABLE_SIZE, [47](#)
 - main, [46](#)
 - measureSearchTime, [46](#)
 - NUM_SEARCHES, [47](#)
 - RANDOM_SEED, [47](#)
 - selectTestKeys, [47](#)
 - TOTAL_SIZE, [47](#)
- BLACK
 - RBTre.hpp, [61](#)
- BSTNode, [13](#)
 - BSTNode, [14](#)
 - key, [14](#)
 - left, [14](#)
 - right, [14](#)
 - values, [14](#)
- BSTree, [15](#)
 - ~BSTree, [16](#)
 - BSTree, [16](#)
 - clear, [16](#)
 - find, [16](#)
 - findNode, [16](#)
 - insert, [17](#)
 - root, [17](#)
- BSTree.cpp, [48](#)
- BSTree.hpp, [48](#)
- buckets
 - HashTable, [26](#)
 - HashTableWithHash< HashFunc >, [27](#)
- clear
 - BSTree, [16](#)
 - HashTable, [24](#)
 - RBTre, [31](#)
- collisions
 - DistributionStats, [18](#)
 - HashTable, [26](#)
 - HashTableWithHash< HashFunc >, [27](#)
- Color
 - RBTre.hpp, [61](#)
- color
 - RBNode, [29](#)
- description
 - HashFunctionWrapper, [23](#)
- DistributionStats, [18](#)
 - avgLoad, [18](#)
 - collisions, [18](#)
 - loadDistribution, [18](#)

- loadFactor, 18
- maxLoad, 18
- nonEmptyBuckets, 18
- totalBuckets, 18
- djbHash
 - benchmark_hash_functions.cpp, 36
- find
 - BSTree, 16
 - HashTable, 24
 - HashTableWithHash< HashFunc >, 27
 - RBTree, 31
- findNode
 - BSTree, 16
 - RBTree, 31
- fixInsert
 - RBTree, 32
- Flight, 19
 - airline, 20
 - arrivalDate, 20
 - arrivalTime, 21
 - Flight, 19
 - flightNumber, 21
 - getKey, 20
 - operator<<, 20
 - operator>>, 20
 - passengers, 21
- Flight.cpp, 49
 - operator<<, 49
 - operator>>, 49
- Flight.hpp, 50
- flightNumber
 - Flight, 21
 - FlightRecord, 22
- FlightRecord, 21
 - airline, 22
 - arrivalDate, 22
 - arrivalTime, 22
 - flightNumber, 22
 - passengers, 22
- func
 - HashFunctionWrapper, 23
- gen
 - generate_data.cpp, 52
- generate_data.cpp, 51
 - AIRLINES, 54
 - gen, 52
 - main, 52
 - PREFIXES, 54
 - RANDOM_SEED, 54
 - randomAirline, 52
 - randomDate, 53
 - randomFlightNumber, 53
 - randomPassengers, 53
 - randomTime, 53
 - saveToCSV, 54
 - UNIQUE_FLIGHT_NUMBERS, 54
- generateFlights
 - benchmark_hash_functions.cpp, 36
 - benchmark_hash_size.cpp, 41
 - benchmark_unique_keys.cpp, 46
 - hash_heatmap.cpp, 56
- generateTableSizes
 - benchmark_hash_size.cpp, 41
- getBucketSizes
 - HashTable, 25
- getCollisions
 - HashTable, 25
 - HashTableWithHash< HashFunc >, 27
- getHashFunctions
 - benchmark_hash_functions.cpp, 37
- getKey
 - Flight, 20
- getTestKeys
 - benchmark_hash_functions.cpp, 37
 - benchmark_hash_size.cpp, 41
- getUniqueKeys
 - main.cpp, 59
- hash
 - HashTable, 25
- hash_heatmap.cpp, 55
 - analyzeDistribution, 56
 - generateFlights, 56
 - HASH_TABLE_SIZE, 56
 - main, 56
 - RANDOM_SEED, 56
 - saveBucketLoadCSV, 56
 - saveTextReport, 56
 - TOTAL_SIZE, 57
 - UNIQUE_KEYS_COUNT, 57
- HASH_TABLE_SIZE
 - benchmark_hash_functions.cpp, 38
 - benchmark_unique_keys.cpp, 47
 - hash_heatmap.cpp, 56
- hashFunc
 - HashTableWithHash< HashFunc >, 27
- HashFunctionWrapper, 22
 - description, 23
 - func, 23
 - name, 23
- HashTable, 23
 - ~HashTable, 24
 - buckets, 26
 - clear, 24
 - collisions, 26
 - find, 24
 - getBucketSizes, 25
 - getCollisions, 25
 - hash, 25
 - HashTable, 24
 - insert, 25
 - tableSize, 26
- HashTable.cpp, 57
- HashTable.hpp, 57
- HashTableWithHash
 - HashTableWithHash< HashFunc >, 27

- HashTableWithHash< HashFunc >, 26
 - buckets, 27
 - collisions, 27
 - find, 27
 - getCollisions, 27
 - hashFunc, 27
 - HashTableWithHash, 27
 - insert, 27
 - tableSize, 28
- insert
 - BSTree, 17
 - HashTable, 25
 - HashTableWithHash< HashFunc >, 27
 - RBTree, 32
- isPrime
 - benchmark_hash_size.cpp, 41
- jsHash
 - benchmark_hash_functions.cpp, 37
- key
 - BSTNode, 14
 - RBNode, 29
- left
 - BSTNode, 14
 - RBNode, 29
- linearSearch
 - main.cpp, 59
- loadDistribution
 - DistributionStats, 18
- loadFactor
 - DistributionStats, 18
- loadFromCSV
 - main.cpp, 60
- main
 - benchmark_hash_functions.cpp, 37
 - benchmark_hash_size.cpp, 42
 - benchmark_unique_keys.cpp, 46
 - generate_data.cpp, 52
 - hash_heatmap.cpp, 56
 - main.cpp, 60
- main.cpp, 58
 - getUniqueKeys, 59
 - linearSearch, 59
 - loadFromCSV, 60
 - main, 60
- maxLoad
 - DistributionStats, 18
- measureSearchTime
 - benchmark_hash_size.cpp, 42
 - benchmark_unique_keys.cpp, 46
- name
 - HashFunctionWrapper, 23
- nonEmptyBuckets
 - DistributionStats, 18
- NUM_SEARCHES
 - benchmark_hash_functions.cpp, 38
 - benchmark_hash_size.cpp, 43
 - benchmark_unique_keys.cpp, 47
- operator<<
 - Flight, 20
 - Flight.cpp, 49
- operator>>
 - Flight, 20
 - Flight.cpp, 49
- parent
 - RBNode, 29
- passengers
 - Flight, 21
 - FlightRecord, 22
- polyHash
 - benchmark_hash_functions.cpp, 37
- PREFIXES
 - generate_data.cpp, 54
- RANDOM_SEED
 - benchmark_hash_functions.cpp, 38
 - benchmark_hash_size.cpp, 43
 - benchmark_unique_keys.cpp, 47
 - generate_data.cpp, 54
 - hash_heatmap.cpp, 56
- randomAirline
 - generate_data.cpp, 52
- randomDate
 - generate_data.cpp, 53
- randomFlightNumber
 - generate_data.cpp, 53
- randomPassengers
 - generate_data.cpp, 53
- randomTime
 - generate_data.cpp, 53
- RBNode, 28
 - color, 29
 - key, 29
 - left, 29
 - parent, 29
 - RBNode, 29
 - right, 29
 - values, 29
- RBTree, 30
 - ~RBTree, 31
 - clear, 31
 - find, 31
 - findNode, 31
 - fixInsert, 32
 - insert, 32
 - RBTree, 31
 - root, 33
 - rotateLeft, 32
 - rotateRight, 33
- RBTree.cpp, 60
- RBTree.hpp, 60
- BLACK, 61

- Color, [61](#)
- RED, [61](#)
- RED
 - RBTre.hpp, [61](#)
- right
 - BSTNode, [14](#)
 - RBNode, [29](#)
- root
 - BSTree, [17](#)
 - RBTre, [33](#)
- rotateLeft
 - RBTre, [32](#)
- rotateRight
 - RBTre, [33](#)
- rsHash
 - benchmark_hash_functions.cpp, [38](#)
- saveBucketLoadCSV
 - hash_heatmap.cpp, [56](#)
- saveTextReport
 - hash_heatmap.cpp, [56](#)
- saveToCSV
 - generate_data.cpp, [54](#)
- selectTestKeys
 - benchmark_unique_keys.cpp, [47](#)
- tableSize
 - HashTable, [26](#)
 - HashTableWithHash< HashFunc >, [28](#)
- TOTAL_SIZE
 - benchmark_hash_functions.cpp, [39](#)
 - benchmark_hash_size.cpp, [43](#)
 - benchmark_unique_keys.cpp, [47](#)
 - hash_heatmap.cpp, [57](#)
- totalBuckets
 - DistributionStats, [18](#)
- UNIQUE_FLIGHT_NUMBERS
 - generate_data.cpp, [54](#)
- UNIQUE_KEYS_COUNT
 - benchmark_hash_functions.cpp, [39](#)
 - benchmark_hash_size.cpp, [43](#)
 - hash_heatmap.cpp, [57](#)
- values
 - BSTNode, [14](#)
 - RBNode, [29](#)